DAY52 — COMPLETE TECHNICAL BLUEPRINT

Project: Consumer Shopping Behaviour Analysis

Document purpose: Convert the approved project direction into an implementation-ready technical blueprint. No production code is included.

Status: Day 2 / Design complete

Design principle: Preserve the existing analytics scope. Where an implementation detail was missing, the simplest practical and free-first option has been assumed.

1. Executive Summary

The project analyses consumer shopping behaviour to identify customer segments, product/category trends, channel performance, decision drivers, repeat-purchase indicators and actionable business recommendations. The existing workflow is treated as the baseline: raw data → cleaning → analysis → visual insights → business dashboard.

The technical design deliberately avoids unnecessary application complexity. The core v1.0 product is an analytics solution rather than a large transactional web application.

2. Finalized Tech Stack

Layer	Selected technology	Reason
Language	Python 3.13.x	Primary analytics language; strong ecosystem and already aligned with project work.
Data processing	Pandas + NumPy	Cleaning, transformation, aggregation and numerical analysis.
Visualization	Matplotlib + Seaborn	EDA charts, distributions, comparisons and correlation analysis.
Database / SQL	MySQL	Practical relational querying, segmentation and aggregation; familiar and widely used.
Dashboard	Microsoft Power BI	Interactive KPI dashboard, slicers, business reporting and stakeholder-friendly presentation.
Notebook / IDE	Jupyter + VS Code	Exploration in notebooks; maintainable project files and documentation in VS Code.
ML (only if required)	Scikit-learn	Classical predictive modelling without introducing unnecessary deep-learning infrastructure.
Version control	Git + GitHub	Free source control, collaboration and project portfolio visibility.
Hosting	Not required for v1.0 analytics deliverable	Avoids unnecessary deployment work. If a static showcase is later needed, GitHub Pages is sufficient.

Authentication: Not required for v1.0 because the core deliverable is an analytics workflow/dashboard, not a multi-user application.

Backend/API: Not required for the core v1.0 analytics workflow. An API should only be added if a future interactive prediction/application layer is approved.

3. System Architecture

Component architecture

```
flowchart LR
    A[Raw Customer Shopping Dataset] --> B[Python / Pandas]
    B --> C[Data Cleaning & Validation]
    C --> D[Feature Preparation]
    D --> E[EDA: Python + Matplotlib/Seaborn]
```

```
C --> F[Clean Analytical Dataset]
F --> G[MySQL]
F --> H[Excel / Pivot Analysis]
F --> I[Power BI]
G --> I
I --> J[Interactive Dashboard]
E --> K[Findings & Recommendations]
J --> K
D --> L[Optional Scikit-learn Model]
L --> K
```

Data flow

1) Ingest raw dataset → 2) profile columns and data types → 3) clean missing/duplicate/inconsistent records → 4) engineer analytical fields → 5) validate quality → 6) store/query structured data in MySQL where required → 7) perform EDA → 8) build Power BI model/dashboard → 9) document findings and recommendations.

Request / interaction lifecycle

For the v1.0 dashboard there is no application request lifecycle. The user interacts with Power BI visuals, slicers and filters; Power BI queries the prepared model/data source and refreshes the visual layer. Python and SQL are upstream analytical preparation tools.

AI interaction

No external AI API is required for v1.0. Predictive analysis, if required by the approved scope, is handled locally with Scikit-learn. This avoids API cost, credentials, rate limits and unnecessary architecture.

External services

GitHub is used for source control. Power BI is used for dashboard delivery. No paid external API is required.

4. Database Design

Relational schema assumption

The analytical database is designed as a compact star-style model. The raw source should remain preserved separately; cleaned/curated tables support SQL analysis and Power BI.

Table	Key fields	Purpose
customers	customer_id PK, age, gender, location	Customer demographic dimension.
products	product_id PK, product_name, category	Product/category dimension.
transactions	transaction_id PK, customer_id FK, product_id FK, purchase_date, quantity, unit_price, discount, rating, payment_method, channel	Central transaction fact table.
purchase_frequency	frequency_id PK, transaction_id FK, frequency_label, frequency_days	Normalized/derived purchase-frequency information if required.

Core constraints

• Primary keys are unique and non-null.

• Foreign keys must reference existing customer/product/transaction records.

• Quantity must be positive; unit price must be non-negative.

• Discount should be constrained to the accepted business range and validated before loading.

• Rating must remain within the dataset's defined rating scale.

• Channel and payment method values should use controlled categories.

• Dates must be valid date values; impossible/future values should be reviewed.

• Duplicate transaction records must be identified before the curated table is created.

User-story validation matrix

Requirement	Data support	Validation
Customer segmentation	customers.age, gender, location	Segment counts, spend and frequency by demographic group.
Category behaviour	products.category + transactions	Revenue, quantity and transactions by category.
Online vs offline	transactions.channel	Channel comparison by volume, revenue and season.
Decision drivers	discount, rating, season/date, payment_method	Grouped comparisons and correlation/association checks; avoid causal claims.
Repeat purchase / retention	customer_id + purchase_date	Count customers with multiple purchase events and repeat intervals.
Recommendations	All curated measures	Every recommendation must trace to an observed metric or pattern.

5. API Design

v1.0 API decision: no API is required for the core analytics product. This is intentional, not an omission. Adding REST endpoints would create backend/authentication/deployment scope without supporting the primary analytics deliverable.

Future-only API candidates, if a web application is later approved:

Endpoint	Purpose	Auth	Validation / errors
GET /api/summary	Return dashboard KPI summary	Future auth	Invalid date/filter → 400; data unavailable → 503
GET /api/customers/{id}	Return customer analytical profile	Future auth	Unknown ID → 404

Endpoint	Purpose	Auth	Validation / errors
GET /api/categories	Return category metrics	Future auth	Invalid filters → 400
POST /api/predict	Future prediction request	Future auth	Schema/type validation → 422; model unavailable → 503

These endpoints are documented as future extension points only; they are not implementation requirements for Day 2.

6. UI & User Flow Design

The primary UI is the Power BI dashboard. Each screen/view exists to answer a business question.

```
flowchart TD
    A[Dashboard Landing / KPI Overview] --> B[Customer & Demographic Analysis]
    A --> C[Product & Category Analysis]
    A --> D[Channel & Payment Analysis]
    A --> E[Decision Drivers & Retention]
    B --> F[Business Recommendations]
    C --> F
    D --> F
    E --> F
```

Screen	Purpose	Main elements
1. Overview	Executive snapshot	Total customers, transactions, revenue, average basket/value, key slicers.
2. Customer Analysis	Understand segments	Age/gender/location distributions, frequency, spend, repeat buyers.
3. Product Analysis	Understand category performance	Category revenue, quantity, rating, top products.
4. Channel & Payment	Compare purchasing channels	Online/offline comparison, payment mix, seasonal patterns.
5. Drivers & Retention	Explore influencing indicators	Discount, ratings, frequency, repeat purchase and relationship views.
6. Recommendations	Turn analysis into action	Evidence-backed marketing, inventory and product recommendations.

Low-fidelity wireframe

```
+-------------------------------------------------------------+
| CONSUMER SHOPPING BEHAVIOUR — OVERVIEW                      |
| [Date] [Channel] [Category] [Gender] [Age]                  |
+-------------------------------------------------------------+
| Revenue | Transactions | Customers | Avg. Value | Repeat %  |
+-------------------------------------------------------------+
| Revenue Trend                | Category Performance         |
|                              |                              |
+------------------------------+------------------------------+
| Customer Segment Summary      | Channel / Payment Mix       |
|                              |                              |
+------------------------------+------------------------------+
| Key Finding 1 | Key Finding 2 | Key Finding 3               |
+-------------------------------------------------------------+
| [Customer] [Products] [Channel] [Drivers] [Recommendations] |
+-------------------------------------------------------------+
```

7. Project Structure

```
consumer-shopping-behaviour-analysis/
■■■ README.md
■■■ ARCHITECTURE.md
■■■ SCHEMA.md
■■■ API.md
■■■ UI-WIREFRAMES.md
■■■ PROJECT-STRUCTURE.md
■■■ IMPLEMENTATION-BLUEPRINT.md
■■■ PROJECT-LOG.md
■■■ data/
■   ■■■ raw/
■   ■■■ processed/
■   ■■■ README.md
■■■ notebooks/
■   ■■■ 01_data_inspection.ipynb
■   ■■■ 02_data_cleaning.ipynb
■   ■■■ 03_eda.ipynb
■   ■■■ 04_analysis_modeling.ipynb
■■■ src/
■   ■■■ data/
■   ■■■ analysis/
■   ■■■ features/
■   ■■■ models/
■■■ sql/
■   ■■■ schema.sql
```

```
■   ■■■ cleaning_checks.sql
■   ■■■ analysis_queries.sql
■■■ powerbi/
■   ■■■ dashboard-notes.md
■■■ reports/
■   ■■■ figures/
■   ■■■ findings.md
■■■ tests/
■■■ requirements.txt
```

Folder responsibilities

• **data/raw:** untouched source files; never overwrite the original dataset.

• **data/processed:** validated and cleaned datasets used by downstream analysis.

• **notebooks:** sequential exploratory and analytical workflow.

• **src:** reusable Python logic that can later be separated from notebooks.

• **sql:** schema, quality checks and reproducible analytical queries.

• **powerbi:** dashboard documentation, measures/KPI notes and refresh instructions.

• **reports:** findings, figures and project evidence.

• **tests:** future data-quality and transformation checks.

• **docs at root:** the Day 2 technical source of truth for implementation.

8. Day 3 Readiness Check

Check	Status	Decision
Scope control	PASS	No web application, authentication or paid AI service added to v1.0.
Data pipeline	PASS	Raw → clean → feature preparation → analysis → dashboard is defined.
Database	PASS	MySQL schema and validation rules are defined where SQL storage/querying is needed.
Dashboard	PASS	Views and navigation are defined.
API	PASS	Explicitly marked unnecessary for v1.0.
ML	CONTROLLED	Only implement if required by the approved project scope; avoid model sprawl.
Deployment	PASS	No hosting dependency for core deliverable.
Day 3 implementation	PASS	Can begin with repository structure, environment, data ingestion and cleaning.

Recommended simplifications

• Do not build a custom login/authentication system for the analytics dashboard.

• Do not build a REST backend unless a web application becomes an approved requirement.

• Do not add multiple ML models merely for comparison; use the simplest defensible model if prediction is required.

• Keep Power BI as the primary stakeholder-facing dashboard.

• Keep raw data immutable and make cleaning reproducible.

9. Day 3 Starting Checklist

• Create/verify GitHub repository.

• Clone repository locally.

• Create the agreed folder structure.

• Create Python virtual environment and requirements file.

• Add raw dataset without modifying the source copy.

• Create initial data-inspection notebook.

• Profile columns, types, nulls and duplicates.

• Record data-quality findings.

• Begin cleaning only after profiling is documented.

10. Day52 Documentation Deliverables

The Day 2 blueprint is organized so the following files can be committed to GitHub: ARCHITECTURE.md, SCHEMA.md, API.md, UI-WIREFRAMES.md and PROJECT-STRUCTURE.md. This PDF is the consolidated human-readable version.

Implementation Blueprint update

The only material Day 2 refinement is to explicitly classify API, authentication, hosting and external AI as **not required for v1.0**. This reduces scope and makes Day 3 implementation immediately actionable. No core business requirement is changed.

11. Git Commit & Push — Manual Task

> Per the project standing rule, GitHub operations must be performed manually with confirmation and screenshots. Do not assume repository creation, cloning, staging, committing or pushing has been completed.

When you reach this stage, the exact commands will be provided one step at a time and we will stop after each manual step for confirmation/screenshot.

12. LinkedIn Progress Post

Day 52 — Turning the project plan into an implementation-ready technical blueprint ■

Today I moved from project planning to technical system design for my Consumer Shopping Behaviour Analysis project. I finalized the technology stack, designed the end-to-end data architecture, defined the analytical database schema, mapped the dashboard user flow, documented the project folder structure, and completed a Day 3 readiness review.

The main focus was keeping the solution practical: Python, Pandas, NumPy, SQL/MySQL and Power BI form the core stack, while unnecessary backend, authentication and paid AI dependencies are intentionally excluded from v1.0.

The goal for tomorrow is simple: stop planning and start building. ■■

#DataAnalytics #Python #SQL #PowerBI #Pandas #DataScience #ProjectDevelopment #GitHub #LearningByBuilding

Assumptions & Traceability Note

This blueprint uses the available project material as the baseline. Where the exact Day 1 technical documents were not available in the working context, reasonable free-first assumptions were made rather than inventing product requirements. The project material explicitly identifies the consumer-shopping problem, objectives and analytics stack, including Python/Pandas/NumPy, SQL, Excel and Power BI. ■filecite■turn4file10■L617-L655■ The presentation also defines the raw-data-to-dashboard workflow and core analytical stages. ■filecite■turn4file14■L992-L1029■